

SMoE: An Algorithm-System Co-Design for Pushing MoE to the Edge via Expert Substitution

Guoying Zhu*, Meng Li^{*✉}, Haipeng Dai^{*✉}, Xuechen Liu*, Weijun Wang[†], Keran Li*,
Jun Xiao[‡], Ligeng Chen[‡], and Wei Wang*

*State Key Laboratory for Novel Software Technology, Nanjing University, [†]Tsinghua University,, [‡]Honor Device Co., Ltd
Email: 522023330124@mail.nju.edu.cn, {meng, haipengdai}@nju.edu.cn, 522025330063@mail.nju.edu.cn,
wangweijun@air.tsinghua.edu.cn, keranli@mail.nju.edu.cn, sunny-xiaojun@hotmail.com,
{chenlg@mail, ww}@nju.edu.cn

Abstract—The Mixture of Experts (MoE) architecture has emerged as a key technique for scaling Large Language Models by activating only a subset of experts per query. Deploying MoE on consumer-grade edge hardware, however, is constrained by limited device memory, making dynamic expert offloading essential. Unlike prior work that treats offloading purely as a scheduling problem, we leverage expert importance to guide decisions, substituting low-importance active experts with functionally similar ones already cached in GPU memory, thereby preserving accuracy. As a result, this design reduces memory usage and data transfer, while largely eliminating PCIe overhead. In addition, we introduce a scheduling policy that maximizes the reuse ratio of GPU-cached experts, further boosting efficiency. Our extensive evaluations show that, compared with state-of-the-art approaches, our method achieves a 48% reduction in decoding latency and maintains an expert cache hit rate above 60%, all while preserving nearly lossless accuracy.

I. INTRODUCTION

MoE architectures offer a promising approach for deploying Large Language Models (LLMs) on edge devices, addressing an increasingly critical need [25, 35, 36]. Edge applications such as smart homes [34], intelligent healthcare [9], autonomous transportation [48], and pervasive video analytics [42] demand low latency and strong data privacy, which makes edge deployment essential. These applications run in a low-batch regime, where latency is more critical than throughput, since an edge LLM usually serves only one device instead of a cluster with high concurrency [20, 29, 40]. Yet, edge servers are often limited in computational capacity and GPU memory, restricting full model deployment and rapid inference [37, 47]. Compared with dense models that compute all parameters for every input, MoE architectures mitigate these constraints by partitioning feed-forward layers into multiple experts [19], activating only a sparse subset per token. This design drastically reduces computational overhead.

However, GPU memory limits force frequent offloading of experts to CPU memory and reloading to the GPU, causing significant inference latency. Since edge GPUs have limited memory and cannot hold all experts simultaneously, inference often requires offloading experts to slower CPU memory or leveraging the CPU to perform computations [23, 26, 50].

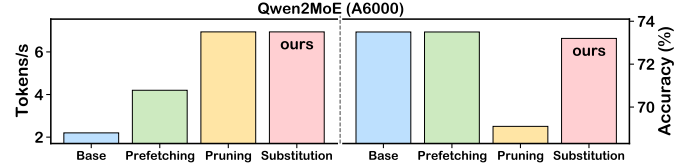


Fig. 1: Token generation speed and accuracy vary among prefetching, expert pruning and our substituting methods.

Both PCIe transfers and CPU computation are significantly slower than GPU execution, introducing 10–100× higher latency. In particular, to mitigate prolonged CPU-GPU expert loading latency, existing solutions fall into two categories: (1) expert prefetching and (2) expert pruning. Prefetching-based works, including MoE-infinity [43], HybriMoE [52], and ProMoE [38], use prefetching instructions to overlap expert loading and computation, hiding CPU-GPU transfer latency. The reduced latency depends on the degree of overlap and is often limited. In contrast, pruning-based approaches [14, 32] reduce latency by directly dropping some experts, which may degrade LLM accuracy due to incomplete expert computation. Moreover, as shown in Fig. 1, to evaluate the effectiveness of these two approaches, we conduct a micro-benchmark on a human examination dataset, including subjects from Math, History, and Biology in the Gaokao benchmark [16]. It can be easily observed that prefetching-based works suffer from high token generation latency, while pruning-based approaches suffer from reduced accuracy. This raises a fundamental yet unanswered question: how can we design a GPU-friendly expert scheduling mechanism that reduces inference latency without compromising model accuracy?

To address this challenge, we propose a novel, third category of methods: expert substitution. This method is motivated by an insightful observation: in fine-grained MoE models like Qwen [6] or DeepSeek [5], although top- k experts may be activated at a given time, only a small subset achieves high scores (termed *top-score active experts*), while the rest receive scores comparable to inactive experts (*low-score active experts*). For these low-score experts, substituting them with other experts having similar scores preserves accuracy or

[✉]Corresponding authors.

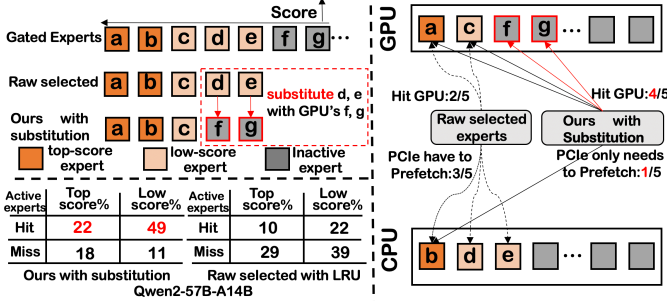


Fig. 2: Naive MoE layer-wise expert loading vs. our substitution-centric expert scheduler.

incurs only limited degradation. This model-level insight opens a system-level optimization opportunity for expert offloading: a low-score activated expert can be replaced with a similar-scoring expert already cached on the GPU. Moreover, this substitution-based method is fully compatible with prefetching-based techniques and can even strengthen them, as it simultaneously reduces the number of experts that need to be prefetched. As Fig. 1 shows, the substitution-based method achieves comparable accuracy with faster token generation than pure prefetching, and comparable generation speed with higher accuracy than pruning on the Gaokao benchmark [16].

To better illustrate our general idea, we provide a simple running example in Fig. 2, which further demonstrates that this strategy largely eliminates the CPU computation burden of low-score active experts while ensuring that top-score active experts are loaded from CPU memory to the GPU in advance. Particularly, by substituting low-score experts d and e with GPU-resident experts f and g of similar scores, the GPU expert hit rate per layer increases from $\frac{2}{5}$ to $\frac{4}{5}$. Besides, this substitution method reduces prefetching overhead from three experts to one and further enhances the GPU expert hit rate to 71% for the Qwen2-57B-A14B model on an A6000 [6].

Challenges. Our work addresses three main challenges:

- How to identify low-score experts and suitable substitutes in GPU memory with minimized accuracy loss?
- How to decide which experts to evict or prefetch, given that cached experts can serve as substitutions, to maximize the overall cache hit rate?
- How to integrate CPU-assisted pipelines to handle experts that can be neither substituted nor prefetched?

Contributions. Our contributions are as follows.

- We introduce the SMoE (Expert Scheduler with Substitution) to minimize decoding latency by maximally substituting low-importance active experts with functionally similar ones already cached in GPU memory, while incurring almost no loss in accuracy.
- We design a CPU-GPU-load pipeline system specifically for MoE LLMs with SMoE, capable of handling online workloads without requiring any offline preparation.
- We extensively evaluate our approach on practical workloads, demonstrating its effectiveness and efficiency.

II. BACKGROUND

Deploying Large Language Models (LLMs) directly on edge devices is crucial for instant, private, secure, and reliable interaction within user environments like smart homes and autonomous vehicles, demanding real-time processing and immediate perception [20, 27, 30]. For LLM on edge, Time Per Output Token (TPOT) [4] is a critical metric that measures the average time between successively generated tokens during decoding. Edge inference typically handles low-batch requests [20], since an edge LLM serves an individual device rather than a computing cluster. In such low-batch scenarios, performance is often constrained by memory bandwidth as in expert offloading, making TPOT optimization essential.

To address the limited GPU memory on edge hardware, MoE LLMs with online expert offloading offer a solution by segmenting traditional layers into specialized experts [19]. This MoE architecture processes only a sparse subset of parameters per input, significantly easing the strain on constrained GPU resources compared to dense networks.

Our goal is to reduce the decoding latency of LLMs with MoE that feature fine-grained expert segmentation on GPU-memory constrained edge devices. We discuss our background from three perspectives: (1) the model architecture (MoE with fine-grained expert segmentation), (2) the online expert offloading architecture, and (3) the metric TPOT (Time Per Output Token), measuring the latency time between successively generated tokens during decoding.

A. LLM with Fine-grained Expert MoE

The MoE LLM substitutes the dense FFN with multiple smaller expert networks [11]. For each input, a router selects the top- k experts to activate, significantly improving parameter efficiency by computing only through these chosen modules.

Unlike traditional MoE that activates 1–2 experts per token, fine-grained MoE divides experts further and activates more per token under similar computational constraints. This promotes specialization and allows richer integration of expert knowledge, enhancing model expressiveness. DeepSeekMoE first introduced this strategy [17], later adopted by models like Qwen2-57B-A14B [6] and XVERSE-MoE-A4.2B [7], achieving strong performance with reduced training costs.

MoE with fine-grained expert segmentation often includes shared experts that process all tokens [18], regardless of routing decisions. This design tackles a major issue in traditional MoE, where common knowledge is redundantly stored. By using shared experts to consolidate common knowledge, the architecture allows other experts to specialize in distinct domains. This division enhances parameter efficiency.

B. Expert Offloading in MoE LLMs

Expert Offloading is crucial for deploying MoE LLMs on edge devices, operating at the expert level rather than the layer level, unlike LLM offloading [39, 49]. It strategically allocates a subset of experts and common parameters, such as attention, token embedding, and router weights to GPU memory, while storing all expert parameters in CPU memory. This approach

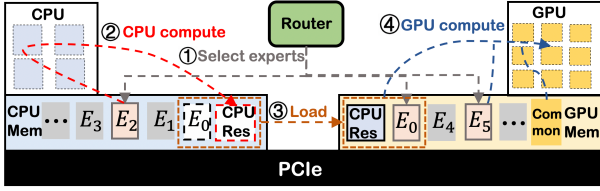


Fig. 3: Online Expert Offloading in MoE LLMs at one layer. Step 1: Router selects the active experts. Step 2: CPU computes part of the active experts in CPU memory. Step 3: Part of active experts and CPU-computed expert results are transferred to GPU memory via PCIe. Step 4: GPU processes experts, consolidating those results with CPU-computed results.

substantially lowers TPOT compared to general LLM methods like llama.cpp [2], which are not tailored for MoE models. Moreover, other techniques such as PowerInfer [39, 44], which target LLMs with ReLU activation functions, do not provide specific offloading strategies for MoE models.

Similarly to the general LLM offloading technique: two main strategies are used, as shown in Steps 2 and 3 of Fig. 3: either freeing up GPU memory to transfer the necessary expert parameters from CPU memory for GPU computation, or directly performing the computations on the CPU and then aggregating the results with those from the GPU. These two approaches can be pipelined. As shown in Fig. 3, while the CPU computes one expert (E_2), another (E_0) can be simultaneously loaded into GPU memory via PCIe. Upon E_2 's completion, its results are then transferred to GPU memory, allowing for concurrent processing and data movement.

Our work addresses unknown edge workloads through online expert offloading. Expert offloading is categorized into online and offline MoE serving strategies, depending on whether experts are dynamically loaded into GPU memory based on the current requests or the entire workload. Online strategies manage dynamically changing edge requests, adapting through flexible scheduling to handle the impact of frequent loading and computation of non-resident experts on latency. Examples include MoE-Infinity [43] and HybriMoE [52]. Offline strategies focus on predetermined workloads, optimizing GPU use by capturing expert activation patterns and employing expert pruning, as seen in MoE-lightning [12].

III. MOTIVATION

Deploying MoE LLMs on edge devices necessitates online expert offloading due to constrained GPU memory. As workloads dynamically shift, so does the set of active experts; however, the limited on-device GPU memory cannot always accommodate all required experts. Consequently, active experts residing in GPU memory are processed directly, while

those in CPU memory are either transferred to the GPU for computation or processed directly on the CPU. However, processing experts not already in GPU memory significantly impacts inference latency, regardless of the offloading method. As Fig. 4 shows, this is primarily because PCIe loading can be one to two orders of magnitude slower than GPU computation, and CPU computation is typically one order of magnitude slower. Thus, deployment of MoE LLMs under GPU memory constraints requires minimizing overhead from expert loading.

Several optimization strategies have emerged for online expert offloading to alleviate bottlenecks. One approach increases the hit rate of experts in GPU memory through prefetching and caching. Prefetching predicts future needs, enabling preemptive loading to reduce TPOT impact [43], while caching strategies like LRU minimize data transfers. Another line of work adopts a related idea by optimizing CPU computation through efficient CPU-side loading pipelines [52].

However, current offloading strategies overlook the significant variation in importance among activated experts, scheduling them uniformly. In MoE architectures, an expert's importance to an input is reflected in its router's gate score, with higher scores indicating greater significance. As Fig. 5 illustrates, a distinct pattern of importance scores emerges among activated non-shared experts: only a few achieve high scores (top-score experts), significantly influencing the output, while others have low scores (low-score experts), similar to inactive experts. This differentiation arises because (1) shared experts handle common knowledge and (2) fine-grained segmentation creates highly specialized non-shared experts. Yet, existing online expert offloading methods, such as prefetching and CPU-load pipelining [52], do not consider this pattern of activated experts on output results. This oversight results in time-consuming operations, like CPU computation and PCIe loading of experts, being used for experts that have minimal impact on the final outcome. However, Table I shows that the low-scoring loading time is the main bottleneck of TPOT.

Our proposed online strategy schedules experts by their importance, substituting low-importance active experts (low-score experts) with functionally similar ones already cached in GPU memory. This mechanism relies on the observation that routing redundancy and noisy gating in sparse MoEs cause tail experts with marginal scores to behave similarly. Previous research indicates that MoE routing exhibits inherent stochasticity because training procedures inject routing noise and enforce load balancing to distribute tokens across the expert pool [19, 36]. Consequently, tail experts within the selected top-k set usually receive marginal scores and demonstrate unstable selection frequencies. These experts generally provide weak or redundant signals. Swapping them with cached alternatives rarely leads to noticeable accuracy drops and may actually prevent the dilution of signals from highly confident experts.

As Fig. 6 illustrates, we first prefetch top-score experts based on predicted expert scores, leveraging pipelining to overlap their loading time with computation. Since PCIe loading time significantly exceeds computation time, as shown in Fig. 4, this prefetching can only ensure timely resource

TABLE I: TPOT time ratio distribution for Qwen on A6000.

Low-score loading	Top-score loading	GPU computing
42%	29%	29%

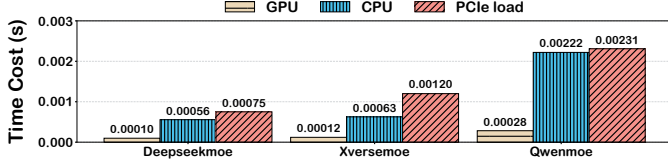


Fig. 4: CPU/GPU (A6000) time for expert-token computing, & PCIe time for expert loading from 3 MoE LLMs.

access for critical top-score experts. After the gating network determines the actual expert scores and the top-score experts (e.g., $k = 3$) are selected, a top-score expert like E_a may already be prefetched while a remaining expert like E_c resides in CPU memory. This typically requires a PCIe transfer or CPU computation. Unlike previous methods, our approach recognizes the low score and minimal output impact of E_c . We therefore substitute E_c with a similarly scored and GPU-resident inactive expert such as E_d . During this process, our scheduler strictly prioritizes the highest-scoring cached candidate whose score falls within a defined threshold. The system specifically attempts to find the closest ranked inactive expert first. If no cached expert satisfies this rigorous score boundary, the scheduler skips the substitution and falls back to either loading the originally selected expert via PCIe or computing it directly on the CPU.

Moreover, although identifying the optimal substitute requires scanning the residing experts in the same layer, this operation incurs no impactful system delay due to the extremely small search space per layer. This lightweight search is executed on the CPU and fully pipelined with ongoing GPU computations and PCIe load operations. This allows almost all the selected experts to be computed directly on the GPU, effectively mitigating the TPOT impact of low-score experts at a negligible cost to accuracy.

IV. EXPERT SCHEDULER WITH SUBSTITUTION

This section outlines the design objectives and approach of our scheduler with expert substitution (SMoE), which aims to achieve three key functions. First, it performs substitution of low-score experts, identifying and replacing them to maximize the GPU expert cache efficiency. Second, it implements top-score expert prefetching to overlap loading and computation times, mitigating accuracy loss from replacement. Third, it

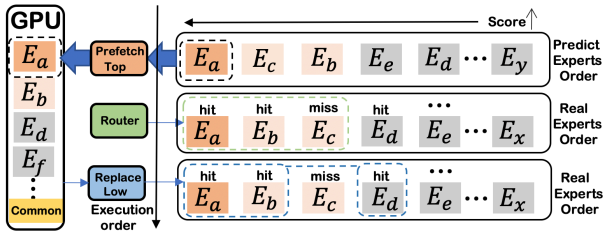


Fig. 6: Our idea: prefetching top-score experts and replacing low-score experts in each iteration at one layer.

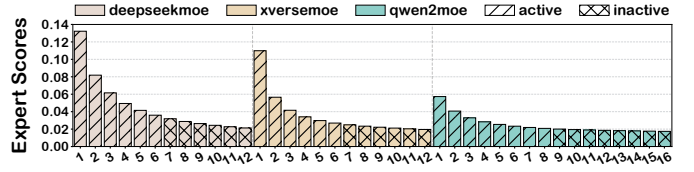


Fig. 5: Average scores at each ranked position across all tokens and layers (sorted in descending order) from 100 prompts.

TABLE II: Definitions.

Symbol	Description
G	Experts already in the GPU.
E_a	Experts selected by top-k.
E_l	Low-score experts.
$E_a \setminus E_l$	Top-score experts.
E_s	Experts in G that can serve as substitutes.
E_p	Experts prefetched from the previous layer.
α	Expert substitution threshold.
S_{k+1}	The gate score of the $k + 1$ -th highest expert.

uses CPU-assisted computing to dynamically decide whether to transfer active experts to the GPU via PCIe or compute them directly on the CPU, addressing prefetching failures and expert-cache router limitations.

A. Design Analysis

As discussed earlier, the absence of experts in GPU memory significantly increases inference latency, which can be mitigated by increasing the hit rate of experts cached in the GPU. Our system improves this hit rate by performing expert substitution independently for each layer, rather than considering all experts at once. This design choice reflects the impracticality of jointly analyzing all layers in transformer LLMs, because the experts chosen in one layer can strongly influence subsequent layers.

Our goal is to maximize, for each layer, the number of routing hits on experts that already reside in GPU memory. To analyze this effect, we define several expert sets and measure their hit counts at the layer level, as shown in Table II. Our optimization objective, given by Equation (1), consists of the number of experts already in the GPU and the number of low-score experts replaced by experts in the GPU.

$$\max |G \cap E_a| + \min(|E_l \setminus G|, |E_s|) \quad (1)$$

It can be expanded as $|(G \setminus E_p \cup E_p) \cap E_a| + \min(|E_l \setminus (G \setminus E_p \cup E_p)|, |E_s|)$, or decomposed into Equation (2).

$$\max |(G \setminus E_p) \cap E_a| + |E_p \cap E_a| + \min(|E_l \setminus (G \setminus E_p)|, |E_l \setminus E_p|, |E_s|) \quad (2)$$

Here, $G \setminus E_p$ and E_a are constant, as they depend on pre-existing expert distributions in the GPU and the top-k selection results by the gate. C_1 , C_2 : Thus, expanding E_l and E_s becomes a viable and beneficial optimization direction. A broader E_s provides more potential candidates for substitution, while a larger E_l increases the pool of GPU-resident substitutes, both directly contributing to raising the hit-related

terms in our optimization objective. Regarding E_p , increasing it can reduce $|E_l \setminus E_p|$ but can increase $|E_p \cap E_a|$. C_5 , C_6 : We increase E_p because the growth in $|E_p \cap E_a|$ directly enhances the overall optimization target, whereas the change in $|E_l \setminus E_p|$ does not necessarily contribute positively to the minimization. While increasing E_p and E_l , we aim to maximize $|E_l \setminus E_p|$, which means minimizing $|E_p \cap E_a|$. C_4 : Therefore, we prioritize prefetching the top-score experts.

However, our system needs to apply constraints to E_l and E_s to balance MoE model with GPU hit rate improvement. The scores by the gate represent an expert's importance to the output, so aligning the scores of the substituted experts with those of the low-score experts can prevent a significant drop in accuracy. We introduce a hyperparameter α (expert substitution threshold) that acts as the score threshold to determine which experts in E_s and E_l are eligible for replacement. Specifically, S_{k+1} is the gate score of the $k+1$ -th highest expert. The constraints are:

$$\begin{aligned} S_{k+1} &< \text{Score}(e) < (1 + \alpha)S_{k+1}, \quad e \in E_l, \\ (1 - \alpha)S_{k+1} &< \text{Score}(e) \leq S_{k+1}, \quad e \in E_s. \end{aligned}$$

These bounds ensure the scores of E_l and E_s are closely aligned with each other. We aim for E_l and E_s to meet these constraints. The constraint on E_p depends on the time available for loading experts from the previous layer's prefetching start to the current layer's expert computation. C_5 : Thus it requires that the cost of prefetching be sufficiently low to save time and allow for the loading of more experts.

Based on the design analysis, our system's implementation criteria are as follows:

- (1) C_1 : **Maximizing** E_l . Low-score experts should constitute a significant proportion of the active experts.
- (2) C_2 : **Maximizing** E_s . The GPU memory must contain an increased inactive experts suitable for substitution.
- (3) C_3 : **Constraints on** E_l and E_s . The scores of low-score experts must be comparable to those of certain inactive experts within the GPU to enable smooth substitution.
- (4) C_4 : **Minimizing** $|E_l \cap E_p|$. Prioritize prefetching top-score experts.
- (5) C_5 : **Maximizing** E_p . Begin prefetching experts for the next layer as quickly as possible to maximize the number of prefetched experts.
- (6) C_6 : **Maximizing** E_p . Ensure the prefetching accuracy.

B. Low-score Expert Substitution

To fulfill these system design requirements, two key *questions* must be addressed: first, how to categorize the active experts selected by the original MoE router, distinguishing between low-score and top-score experts to meet C_1 , C_3 ; and second, how to design a GPU cache strategy that retains a greater number of experts eligible for substitution within the GPU, aiming to meet C_2 . We design an expert-cache router and a cache eviction to answer these two questions.

Expert-cache router. Algorithm 1 illustrates our approach for the expert-cache router. Those experts with scores above

Algorithm 1 Expert-Cache Router

Require: α, k, S ▷ Score inputs
Ensure: O ▷ Output cacherouter_experts
Initialize O , set thresholds T, L, R , sets A, B, C ▷
 $A = E_s, B = E_l, C = E_a \setminus E_l$
for each token t **do**
Sort S_t desc; $S_{k+1} \leftarrow (k+1)$ -th score of S_t
 $T \leftarrow (1 + \alpha)S_{k+1}; L \leftarrow S_{k+1}; R \leftarrow (1 - \alpha)S_{k+1}$
for each expert e **do**
if score of $e > T$ **then** Add e to $O[t]$ and C
end if
end for
Initialize B_t, A_t
for each expert e **do**
if $L \leq \text{score of } e < T$ **then** Add e to B_t
else if $R \leq \text{score of } e < L$ and e in GPU or C **then**
Add e to A_t
end if
end for
if $|A_t| \geq |B_t|$ **then**
Select top $|B_t|$ experts from A_t ; Add to $O[t]$
else
Add A_t to $E[t]$; Add top $|B_t| - |A_t|$ experts from B_t to $O[t]$
end if
end for
return E

$(1 + \alpha)S_{k+1}$ are top-score experts, while scores between S_{k+1} and $(1 - \alpha)S_{k+1}$ are low-score. For top-score experts, when we process multiple requests concurrently, multiple tokens are decoded, each possessing its own set of top-score experts at the current layer. Given their critical role in computation, these experts are retained in the results of the expert-cache router and called as the top-score expert set. For low-score experts, if they are already in GPU memory or part of the top-score expert set, they are retained in the expert-cache router's results without incurring additional CPU overhead. Otherwise, they can be replaced with an inactive expert whose score falls between $(1 - \alpha)S_{k+1}$ and S_{k+1} and is present in the GPU or in the top-score expert set. We select the $|E_l \setminus G|$ highest-scoring alternatives from E_s to be included in the expert-cache router's results. If $|E_l \setminus G|$ is larger than $|E_s|$, then only $|E_l \setminus G| - |E_s|$ low-scoring experts are prepared for loading into GPU memory or for direct computation by the CPU.

Cache eviction. The cache eviction policy is instrumental in managing the removal of experts from GPU memory when new experts are loaded. As depicted in Fig. 7, experts that achieve higher scores, even if they are inactive in a particular iteration, have a greater inherent probability of being reused in the next 3 iterations, either as top-k or alternative experts, compared to those with minimal scores. This indicates that experts with high scores in the current iteration have consistently achieved high scores in recent decoding iterations. To preserve experts in the GPU with scores similar to those of active

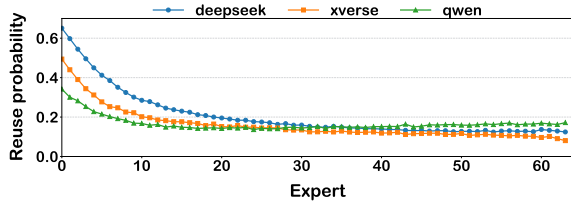


Fig. 7: The reuse probability of experts based on score (in descending order) in three MoE models.

low-score experts, we employ a score-based strategy. This strategy involves evicting the expert that exhibits the lowest average activation score accumulated over the preceding n iterations. Specifically, for an expert i , its score at the j -th token generation during decoding is denoted as $S_{i,j}$. With m experts in total, the expert to be evicted is the one with the minimum average score calculated as follows:

$$\text{Evicted expert} = \arg \min_i \frac{\sum_{k=\max(1, j-n)}^j S_{i,k}}{j - \max(1, j-n) + 1} \quad (3)$$

This approach prioritizes the retention of experts that have demonstrated a higher historical impact, as measured by their contribution to model outputs, over a strategy that relies solely on the recency of access, such as Least Recently Used (LRU). Crucially, our score-aware eviction policy takes into account the activation scores of all experts accessed within the observation window of n iterations. This includes inactive experts that can serve as substitutes for low-score experts, ensuring that the GPU retains a pool of experts that are both relevant and likely to enhance future computations.

Moreover, to prevent evicting an expert immediately before its use, the system dynamically elevates the eviction priority of any expert selected by the expert router for computation in a layer. This temporary protection shield ensures the expert remains resident in GPU memory throughout its required computation window. The shield is automatically revoked upon completion of the layer’s computation, returning the expert to standard eviction eligibility based on its score history.

C. Loading Top-score Expert Online

Our prefetching prioritizes top-score experts to meet C_4 . This technique effectively overlaps the latency of expert loading with ongoing computation time. Additionally, we aim to minimize the cost of prefetching to meet C_5 and enhance the accuracy of prefetching to meet C_6 .

As shown in Fig. 8, the prediction process is carried out entirely with the parameters in GPU. Due to the GPU’s superior speed, it tends to be more idle compared to the CPU and PCIe. As a result, the time taken by the GPU for prediction is often overshadowed by the time required for calculations by the CPU and loading by PCIe. Consequently, the time cost of prediction on TPOT is rather minimal.

Moreover, we can ensure the accuracy of prefetching. We perform calculations on both unshared experts currently

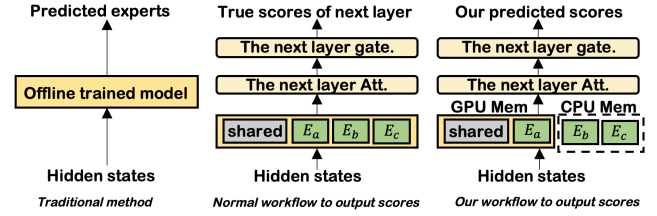


Fig. 8: Our prefetching method compared to traditional method and normal workflow to output true scores.

in the GPU memory and shared experts, generating hidden states. These hidden states are then processed using the next layer’s key-value cache to complete the attention computation. Subsequently, we carry out a gate computation to determine the scores for all experts in that layer. The gate computation results, derived from calculations involving the shared experts and experts in GPU memory, are more accurate for two main reasons. First, shared experts process universal information across all inputs and remain constantly available in the GPU memory, which enhances the accuracy of the computations. Second, our cache eviction strategy, as detailed in Section IV-B, ensures that high-scoring and thus important experts are retained in the cache during the most recent decoding stages. These factors collectively lead to score results that closely align with the true outcomes. If a prediction is wrong, the system simply reverts to the baseline state without performance penalties. We immediately clear mispredicted experts from the load queue and insert the correct ones.

While prior works like HybriMoE [52] utilize prefetching, their approach predicts multiple future layers and attempts to prefetch all absent experts. Because PCIe loading is significantly slower than computation, this untargeted approach creates excessive memory bandwidth pressure. Consequently, the system often fails to prefetch all required experts in time, bottlenecking the pipeline. In contrast, we classify experts into top-score and low-score via the gating network and strictly prefetch only the top-score experts. This targeted approach drastically reduces PCIe load pressure, ensuring a realistic and effective overlap between compute and fetch times. Furthermore, predicting only a small subset of top-score experts inherently yields higher prediction accuracy because top-score experts remain high-impact on the output of the next layer even with incomplete expert computation.

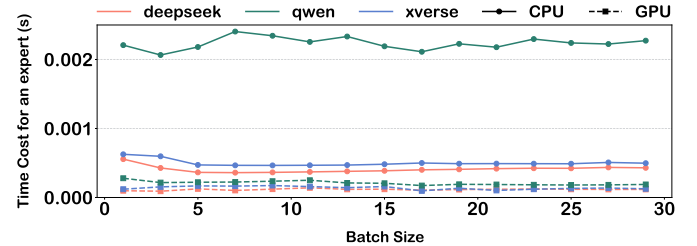


Fig. 9: GPU (A6000) vs CPU (8-core) at low batch.

Algorithm 2 CPU-assisted Task Load Scheduling

Require: S, C_{CPU}, C_{load}

Ensure: n_{load}, n_{CPU}

```

1:  $S$ : List of experts  $\{uid_i\}$  sorted in descending order by
   score
2:  $T_{CPU} \leftarrow 0, T_{load} \leftarrow 0, n_{load} \leftarrow 0, n_{CPU} \leftarrow 0$ 
3:  $l \leftarrow 0, r \leftarrow |S| - 1$ 
4: while  $l \leq r$  do
5:   if  $T_{load} \leq T_{CPU}$  then
6:      $T_{load} \leftarrow T_{load} + C_{load}, n_{load} \leftarrow n_{load} + 1$ 
7:      $l \leftarrow l + 1$ 
8:   else
9:      $T_{CPU} \leftarrow T_{CPU} + C_{CPU}, n_{CPU} \leftarrow n_{CPU} + 1$ 
10:     $r \leftarrow r - 1$ 
11:   end if
12: end while
  
```

V. SYSTEM IMPLEMENTATION

This section details the implementation for efficient GPU-CPU coordination in expert-offloading. Despite improvements in the GPU expert cache ratio via low-score substitution and top-score prefetching, some experts still need to be transferred from CPU to GPU due to prefetching failures from high demand or low PCIe bandwidth, and predictive errors. The expert-cache router may also fail to replace all low-score experts in GPU memory, particularly with significant score differences from inactive experts. To address these issues, we introduce CPU-assisted task load scheduling to balance CPU and GPU loads, boosting system efficiency. Additionally, we give a pipeline example between Layers during SMOE decoding with CPU-assisted computing to show how our design schedules tasks to reduce execution bubbles.

A. CPU-assisted Task Load Scheduling

The CPU-assisted task load scheduling system is designed to dynamically decide whether to transfer activated experts, which are not pre-cached in GPU memory, to the GPU via PCIe for computation or to perform the calculations directly on the CPU. This decision-making process is guided by Algorithm 2, which uses a two-pointer strategy to balance the cumulative costs associated with loading and CPU computation times. In addition to balancing these costs, we prioritize loading high-scoring experts into GPU memory because, as shown in Fig. 7, these experts are more likely to be reused.

The scheduling system aims to minimize idle times by balancing expert loading time, $T_{load} = n_{load} \times C_{load}$, with CPU computation time, $T_{CPU} = n_{CPU} \times C_{CPU}$. Here, C_{load} is the time to transfer an expert via PCIe, and C_{CPU} is the average time for a CPU computation. The system uses the past p instances of these times to optimize scheduling. C_{CPU} is treated as a constant due to stable CPU times across low-batch sizes, as shown in Fig. 9, simplifying our model. Fig.

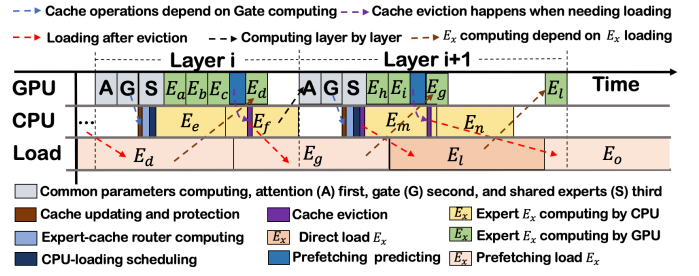


Fig. 10: SMOE pipelines GPU, CPU, and load operations between two MoE layers.

4 indicates minimal GPU operation times compared to CPU and loading times, allowing us to approximate the total time by C_{load} alone. Thus, C_{load} is the primary cost factor, with C_{CPU} as a constant. The optimization goal is expressed by the following equation, which seeks to minimize the maximum of T_{load} and T_{CPU} :

$$\min \max(n_{load} \times C_{load}, n_{CPU} \times C_{cost}) \quad (4)$$

Unlike frameworks that offload heavy computation to the CPU, SMOE's substitution-centric design minimizes CPU involvement. This not only supports a wider range of legacy/edge CPUs lacking specialized instructions but also eliminates the sensitivity to batch-size fluctuations inherent in CPU-based computation.

B. Pipeline Example between Layers

Fig. 10 illustrates the SMOE pipeline, which orchestrates GPU, CPU, and PCIe resources across consecutive layers (i and $i+1$). Colored blocks denote distinct operation phases. Dashed arrows explicitly indicate critical data dependencies, which include: (1) cache updates and protection shield activation are triggered strictly after Gating and attention; (2) GPU expert computation initiates only after full data loading; and (3) layer $i+1$ execution is serialized after layer i . While these dependencies introduce execution bubbles, our pipeline design effectively masks these overheads within the dominant PCIe transfer latency.

CPU Operations: The CPU operations are divided into four key parts, all integral to the SMOE's functionality and the computation of expert parameters on the CPU:

- (1) **Expert-cache router calculation:** This part replaces some low-score experts from layer i with inactive experts in GPU.
- (2) **CPU-assisted task load scheduling:** This process determines which experts should be computed directly on the CPU and which should be loaded into the GPU.
- (3) **CPU expert computation:** CPU computes the parameters of experts from layer i not in GPU.
- (4) **Cache eviction and Protection Shield:** Cache eviction decides which expert to evict when loading new experts into the GPU. Protection shield maintains the experts in the current layer to prevent evicting an expert immediately before its use. Both of these operations can be overlapped by GPU computing and expert loading by PCIe.

GPU Operations: The GPU operations are divided into four parts, three of which focus on computing expert parameters from layer i , while one is dedicated to prefetching parameters from layer $i + 1$:

(1) **Common parameter computation:** This part computes the attention and gate parameters within the common parameters of layer i on the GPU.

(2) **Direct expert computation:** This part computes experts that are already in the GPU.

(3) **Expert Prediction for Prefetching:** This process predicts which experts from layer $i + 1$ should be prefetched.

(4) **New Expert Computation:** This part continues the computation of newly loaded experts once the PCIe loading process is complete.

PCIe Load Operations: The PCIe load operations are divided into two parts:

(1) **Prefetching Experts from Layer $i + 1$:** This part prefetches the necessary experts from layer $i + 1$ to the GPU in advance.

(2) **Loading Experts from Layer i :** This part loads the experts from layer i into the GPU immediately as needed.

C. Selecting value for hyperparameter α

Within our scheduler, the hyperparameter α defines the ratio of experts to be substituted, controlling the tradeoff between accuracy and token generation speed. An appropriate α identifies closely scored experts, including low-score and GPU-resident cached experts, that can be safely substituted with minimal accuracy loss. A larger α reduces latency by allowing more low-score experts to be replaced, but it may increase the score gap between substituted and cached experts, potentially amplifying output deviation. Besides, the α selection is also model/workload dependent. To address this, we formalize it as the following constrained optimization problem:

$$\min_{\alpha} A(\alpha) \quad \text{s.t.} \quad T(\alpha) \leq R, \quad (5)$$

where $A(\alpha)$ denotes the accuracy loss, $T(\alpha)$ is the average TPOT, and R is the TPOT budget specified by the user. Here, we assume that the user has a latency requirement while aiming to maximize accuracy. To solve this, we first observe that the latency metric (or TPOT) generally decreases linearly with the value of α . Intuitively, a larger α allows more low-score experts to be substituted, which increases the computation performed on the GPU and hence decreases the TPOT. Based on this observation, as shown in Fig. 11, we can approximate $T(\alpha)$ by fitting a low-degree polynomial to the empirical relation between α and $T(\alpha)$. This yields a smooth and simple function that can be evaluated efficiently. Given a target latency constraint R , we then perform a simple one-dimensional search over α and select the smallest value whose predicted $T(\alpha)$ satisfies the constraint, thereby ensuring minimal accuracy loss while meeting the latency requirement.

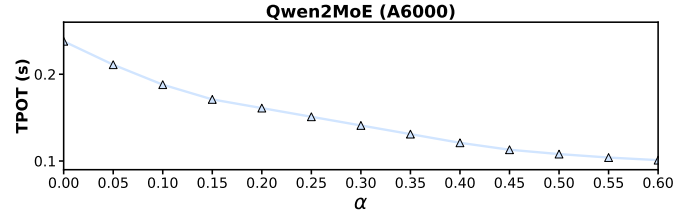


Fig. 11: Relationship between α and $T(\alpha)$.

TABLE III: Model and GPU Configurations

Setting	Model	GPU
S1	deepseek-moe-16b [5] (31GB)	3080ti (12GB)
S2	XVERSE-MoE-A4.2B [7] (49GB)	4060ti (16GB)
S3	Qwen2-57B-A14B [6] (107GB)	A6000 (48GB)

VI. EVALUATION

This section aims to demonstrate that our method, SMOE, significantly reduces the latency of token generation during the decoding phase, increases the hit rate of experts within the GPU, and causes almost no loss in accuracy. Additionally, we conduct experiments to illustrate the impact of various components (expert-cache router, cache eviction strategy, prefetching, and CPU-assisted task load scheduling) on decoding performance and to analyze the reasons behind these impacts.

A. Setup

In designing our experimental setup for LLM inference on edge devices, we focus on three key aspects. First, we select models exceeding edge GPU memory capacity to evaluate performance under typical resource constraints, reflecting real-world scenarios [15]. Second, we employ popular models with a fine-grained MoE architecture, effective as shown in recent studies [5]. Third, we test diverse model types and GPU configurations to ensure result robustness and reliability. We assess three experimental settings, detailed in Table III. Additionally, since MoE expert selection depends on workload, we test various workload types to show our method enhances inference speed without significant accuracy loss. Finally, we demonstrate our approach's superiority by comparing it with both the most advanced and popular methods, using metrics such as TPOT, TTFT, accuracy, and GPU expert cache ratio.

Models. We assess the performance of three widely recognized MoE models featuring a fine-grained MoE architecture: deepseek-moe-16b-base [5], Qwen2-57B-A14B-Instruct [6], and XVERSE-MoE-A4.2B-Chat [7]. Given that the objective of this study is to facilitate lossless models, we do not run quantized models. However, it is worth noting that our approach is orthogonal to quantization operations, and thus, it can also be applied to serve quantized models effectively.

Hardware. We test on a single NVIDIA RTX 3080 Ti GPU (12GB), a single NVIDIA RTX 4060 Ti GPU (24GB), and a single NVIDIA A6000 GPU (48GB). We evaluated SMOE on two setups: (1) Edge/Legacy: RTX 3080Ti / 4060 Ti on PCIe

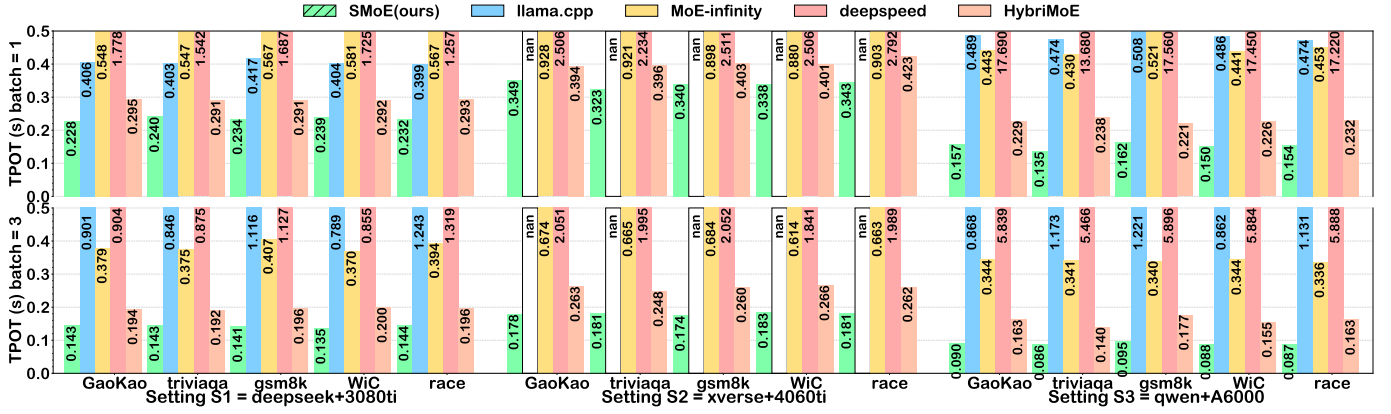


Fig. 12: TPOT of four baselines and our method in five workloads.

TABLE IV: Generation quality across workloads and models at various expert substitution thresholds α .

Workloads	Model	0.0	0.05	0.1	0.15	0.2	0.25	0.3	0.35	0.4	0.45	0.5	0.55	0.6
GaoKao Acc. (%)	Deepseek	27.2	27.6	28.2	29.5	29.3	28.9	28.1	27.7	27.3	26.8	26.7	26.2	25.9
	Xverse	47.2	47.5	47.9	48.1	49.0	47.2	47.9	47.5	46.5	46.8	47.5	46.2	45.9
	Qwen	73.5	74.2	74.8	75.8	76.0	76.1	74.2	74.1	73.2	72.1	71.6	70.7	71.7
WiC Acc. (%)	Deepseek	50.7	50.9	51.3	51.6	50.9	51.7	50.6	50.7	51.9	51.8	51.7	50.1	50.4
	Xverse	50.0	50.1	50.3	50.2	50.2	50.0	50.0	50.2	50.2	50.1	50.00	49.8	49.8
	Qwen	60.5	60.7	60.5	60.6	60.9	60.7	60.6	60.7	61.9	60.8	60.3	60.4	60.1
Triviaqa Acc. (%)	Deepseek	59.3	59.1	59.3	58.5	58.5	57.7	59.6	59.0	58.9	58.7	58.6	57.4	57.6
	Xverse	53.1	53.2	52.8	52.7	53.1	53.3	53.4	53.8	52.8	53.1	53.2	52.1	52.4
	Qwen	69.3	69.2	68.7	68.5	68.5	67.7	69.6	69.0	68.9	68.7	68.6	67.6	67.9
Race-mid Acc. (%)	Deepseek	70.0	69.8	69.2	69.7	69.9	70.4	70.0	69.1	70.2	70.2	70.4	68.9	68.6
	Xverse	81.4	81.6	81.8	81.0	82.3	82.0	81.9	80.9	82.9	82.3	82.4	81.1	80.3
	Qwen	80.0	79.8	79.9	79.7	79.9	80.4	80.0	79.1	80.2	80.1	80.4	79.8	79.6
Gsm8k Acc. (%)	Deepseek	51.4	51.6	51.2	51.4	49.5	51.9	51.0	50.2	49.2	48.8	47.8	47.9	48.8
	Xverse	62.9	62.7	62.9	62.4	63.3	61.7	62.5	61.2	61.2	60.3	60.4	60.1	58.6
	Qwen	85.7	85.5	85.7	85.2	85.8	85.6	85.1	85.3	84.5	84.1	83.7	83.2	83.3
MT-bench Score (1-10)	Deepseek	1.43	1.42	1.58	1.53	1.59	1.47	1.39	1.41	1.43	1.39	1.45	1.20	1.50
	Xverse	6.41	6.30	6.17	5.91	6.17	6.08	5.79	6.11	6.15	5.81	5.50	5.61	5.70
	Qwen	8.52	8.43	8.39	8.39	8.40	8.37	8.33	8.21	8.28	8.29	8.36	8.23	8.19
MMLU Acc. (%)	Deepseek	38.0	39.1	39.8	39.2	39.4	39.4	39.8	38.9	39.2	39.1	39.2	37.2	36.1
	Xverse	39.4	40.3	43.1	37.8	42.3	41.9	42.3	41.7	42.1	43.9	45.2	46.3	42.9
	Qwen	71.0	70.8	70.0	70.3	70.6	71.0	69.8	68.8	71.0	67.8	71.0	71.0	70.4
HumanEval pass@1(%)	Deepseek	26.7	26.5	27.4	26.3	25.0	24.1	26.2	22.7	23.7	22.4	21.3	21.6	20.7
	Xverse	46.3	48.7	51.2	49.6	48.2	47.3	52.4	51.9	50.0	49.7	52.4	48.3	47.6
	Qwen	53.0	51.8	51.2	50.7	50.0	53.3	56.7	53.4	50.6	50.9	50.6	48.7	47.6

3.0 with Intel E5-2683 v3, and (2) High-end: NVIDIA A6000 on PCIe 4.0 with Intel Xeon Gold 6444Y.

Workloads. We select these datasets to guarantee high diversity across evaluation tasks and knowledge domains. Gaokao [16] and MMLU [22] cover structured academic knowledge from secondary to college levels, including mathematics, science, social science and computer science. TriviaQA [24] and RACE-mid [28] test open-domain QA and reading comprehension. WiC [33] evaluates contextual semantic understanding. GSM8K [10] measures multi-step mathematical reasoning. MT-Bench [51] assesses conversational quality and instruction following. HumanEval [13] evaluates code generation capability. This broad coverage enables a comprehensive and robust evaluation of our model. We use the

Math_I, Math_II, History, and Biology datasets in the Gaokao benchmark [16] and College_computer_science, Management, International_law and Logical_fallacies in the MMLU [22].

Metrics. We use three metrics to test inference speed: TPOT, TTFT and GPU cache ratio. For decoding performance, we evaluate the TPOT as the key metric for the decoding stage. For prefilling performance, we assess the TTFT for the prefilling stage. Additionally, we use the GPU expert cache ratio to reflect the GPU memory utilization efficiency, which is determined by the expert scheduling strategies of different methods. We use three metrics to test model performance: Accuracy, GPT-4 Score, and pass@1. Specifically, OpenCompass [3] serves as a comprehensive framework to assess accuracy across diverse datasets, including Gaokao [16],

triviaQA [24], WiC [33], Race-mid [28], gsm8k [10], and MMLU [22]. For code generation capabilities, we evaluate the HumanEval dataset [13] using the pass@1 metric, which measures the percentage of problems solved correctly on the first attempt. Additionally, MT-Bench (GPT-4 Score) is designed to evaluate the multi-turn conversational and complex instruction-following abilities of LLMs through challenging open-ended prompts. This is to demonstrate that the impact of our strategy on accuracy is minimal.

Baselines. We evaluate our work, comparing it against baseline systems that support running MoE LLMs without enough GPU memory on the local platform: (1) *MoE-infinity* [43] is designed for efficient MoE inference on personal machines with limited GPU memory. It leverages the high activation sparsity in the decode phase, where few experts are reused often. (2) *Llama.cpp* [2] is a C++ implementation enabling efficient LLM inference on CPUs, optimized for low GPU memory devices. (3) *DeepSpeed* [8] optimizes large model training and inference by loading transformer layers onto the GPU layer-wise, enabling it to manage GPU-memory constraints without storing the entire model on the GPU. (4) *HybriMoE* [52] is an offloading system designed to enhance resource utilization via a novel CPU-GPU scheduling and cache management framework. As HybriMoE is built on the ktransformer architecture [1] with quantization, we’ve modified our implementation to remove these effects for fair evaluation comparisons.

B. Overall Performance

The overall performance section highlights our method’s superior decoding performance compared to baselines, particularly at low batch sizes of 3 and 1, typical for edge deployment. We assess TTFT for prefiling performance at batch=1, demonstrating that our method matches or exceeds other approaches during the prefiling stage. To reflect real-world workload variability, we sample about 1000 data points from each of five datasets and use basic prompts from openCompass [3]. Note that due to the Xverse model’s incompatibility with the GGUF format, llama.cpp cannot run the S2 Xverse model, so we use nan to denote its value in our experiments.

Decoding performance. Our method consistently achieves a 24% reduction in TPOT compared to the best existing approach on average at batch=1, and a 35% reduction at batch=3, as shown in Fig. 12. Notably, our method performs better when the batch size is 3 than when it is 1. This can be

attributed to two key factors. First, in multi-batch scenarios, GPU computation time does not change significantly as the batch size increases, as shown in Fig. 9. Moreover, since more experts need to be loaded when decoding multiple tokens at a time, the proportion of PCIe load in TPOT increases. Consequently, our strategies aimed at improving expert utilization in GPUs and reducing PCIe load become more effective. Second, we can replace the low-score experts of a certain token with the top-score experts of other tokens in the same batch. Since top-score experts are inevitably computed, this approach avoids the need to additionally load the low-score experts, thus enhancing TPOT performance when batch=3.

Notably, our method significantly reduces TPOT under S3 settings, achieving a 48% reduction on average at batch=3 compared to the best baseline, and 34% reduction at batch=1. This is because the A6000 GPU boasts high computational speed, so TPOT in these settings is dominated by PCIe load time (in the case of MoE-infinity and HybriMoE) and CPU computation time (in the case of llama.cpp, DeepSpeed and HybriMoE). By improving expert utilization in GPU memory, our approach effectively reduces such CPU computations and PCIe loads. In contrast, performance gains in S1 are less pronounced, with a 27% reduction on average at batch=3 compared to the best baseline, and a 20% reduction at batch=1. This is because the computational capability of the 3080Ti GPU offers no significant advantage over CPU, which limits the benefits of improved GPU memory efficiency.

GPU expert cache ratio performance. As shown in Fig. 13, our method consistently achieves at least a 65% improvement in cache ratio across all tested settings compared to the best existing approach. This confirms that our method directly enhances the utilization of experts in GPU memory. In contrast, methods like llama.cpp and deepspeed implement offloading but are not designed for dynamic workloads, leading to static expert caching in the GPU that fails to adapt to workload changes. Due to deepspeed’s sequential loading of layers into the GPU for computation, we don’t conduct cache ratio experiments on it. MoE-infinity [43] employs a prefetching strategy that can increase the hit rate of experts in the GPU based on the current workload, but it requires historical router data, which can lead to decreased prefetching accuracy when records are incomplete. Moreover, prefetching alone is insufficient to load all active experts.

Accuracy performance. Evaluations conducted with our benchmarks, across diverse domain datasets show that our low-score expert substitution method causes only negligible accuracy variation when the substitution threshold is below 0.35, and even improves accuracy in some cases, as shown in Table IV. This occurs because, in MoE models, top-score experts are highly influential, while fluctuations in low-score expert contributions have little effect on the final output as long as a sufficient number of experts are retained. To further enhance inference efficiency on edge devices, we favor a higher substitution ratio while maintaining this minimal loss. This strategy classifies more experts as low-score, increasing

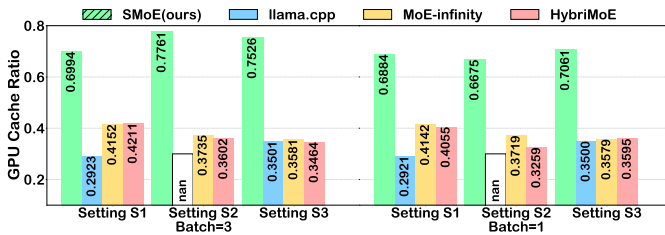


Fig. 13: GPU cache ratio on average.

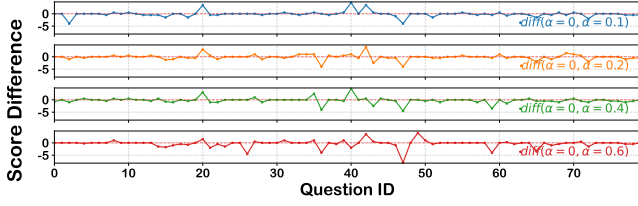


Fig. 14: Correlation between (α) and MT-bench score.

substitution opportunities and reducing expensive PCIe data transfers. In practice, we set the substitution threshold to 0.35 in S1, 0.3 in S2, and 0.25 in S3.

As Table V shows, SMoE maintains the highest fidelity, performing closest to the original model. The expert-skipping method [32] yields the second-best performance, whereas quantization experiences a significant accuracy degradation. Specifically, the expert-skipping baseline directly discards the experts that would otherwise be substituted in SMoE. Consequently, the number of activated experts per token varies dynamically, with higher substitution rates leading to more dropped experts. Meanwhile, the GPTQ baseline quantizes a subset of layers to INT8, which is deliberately configured to achieve a GPU cache hit rate comparable to that of SMoE for a fair comparison.

Fig. 14 demonstrates that a higher substitution rate correlates with increased accuracy variance. The plotted line tracks the difference in MT-Bench scores between SMoE (at the current α) and the original qwenmoe, where larger fluctuations signify greater deviation from the baseline. The maximum variance is concentrated in questions 40–50, which correspond to the coding domain. Despite this, Fig. 15 confirms that our method achieves highly competitive performance across all domains, with only a slight performance penalty.

Prefilling performance. Our method consistently achieves a 11% reduction in TTFT compared to the best existing approach on average, as shown in Fig. 16. While our method still outperforms others, this improvement stems from the pipelining strategy across CPU, GPU, and loading processes rather than enhanced expert utilization in GPU memory. The prefilling stage differs from the decoding stage: expert activation in decoding is relatively sparse, so improved expert utilization in GPU memory can enhance overall TPOT. In

TABLE V: Performance comparison of Qwen2MoE.

Benchmarks	Methods			
	Original	Skipping	Quantization	SMoE($\alpha = 0.25$)
GaoKao Acc. (%)	73.5	69.1	58.9	76.1
WiC Acc. (%)	60.5	60.66	59.8	60.7
triviaqa Acc. (%)	69.3	67.9	62.1	67.7
Race-mid Acc. (%)	80.0	80.0	78.9	80.4
gsmk Acc. (%)	85.7	83.6	81.7	85.6
MTbench (1-10)	8.52	7.40	8.01	8.37
MMLU	71.0	70.2	51.1	71.0
HumanEval pass@1(%)	53.0	53.0	31.1	53.3

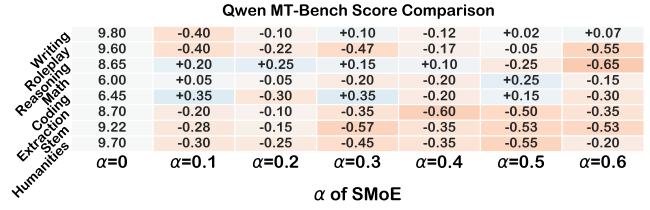


Fig. 15: Per-domain Analysis of SMoE.

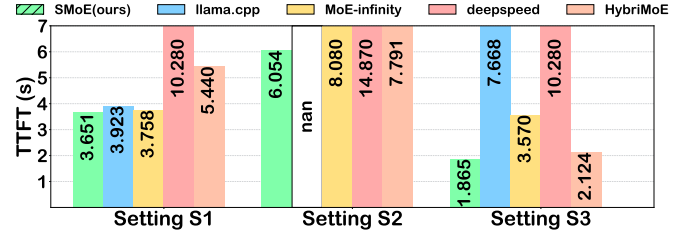


Fig. 16: Prefilling time of baselines and SMoE on average.

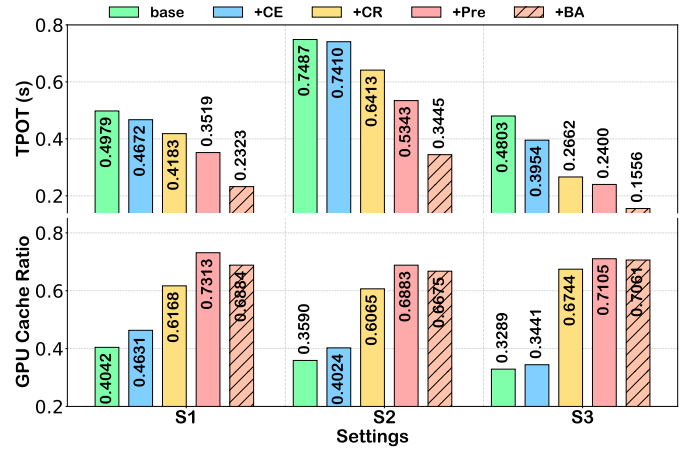


Fig. 17: Impact of components on TPOT and cache ratio.

contrast, expert activation during prefiling is inherently dense, with nearly all experts in the GPU requiring computation. Thus, increasing expert utilization in GPU memory yields limited gains here.

C. Ablation Studies

Fig. 17 analyzes the role of each component in our method in enhancing overall performance via a stepwise incremental

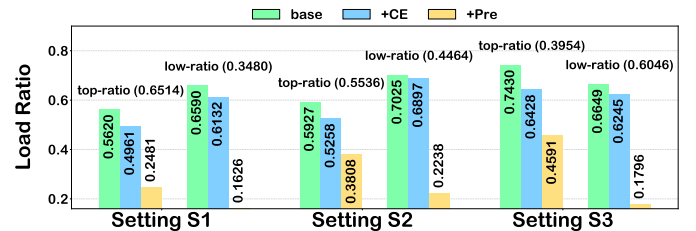


Fig. 18: Fewer top/low-score experts loading.

approach (batch size = 1), with each step reducing TPOT. The baseline uses LRU expert offloading, but lacks an expert-cache router, prefetching, and our caching strategies.

Cache eviction analysis. Compared to the baseline, this reduces TPOT by an average of 8% across settings and increases GPU cache ratio by 11% compared to LRU. We add a cache-eviction strategy (labeled +CE), which retains recently high-scoring experts during decoding to facilitate replacing those with the lowest score. As shown in Fig. 18, the cache-eviction strategy reduces the probability that active top-score and low-score experts are not in GPU memory, with a more notable improvement in top-score expert hit rates. This is because top-score experts, which achieve high scores in the current layer, only occasionally have significantly low scores in the recent few tokens; thus, the cache-eviction strategy prioritizes retaining them. As shown in Fig. 19, even ignoring its benefits to the replacement policy, the cache-eviction strategy outperforms the three traditional caching methods.

Expert-cache router analysis. Relative to +CE, this lowers TPOT by 20% and improves GPU cache ratio by 60%. Building on +CE, we introduce an expert-cache router (labeled +CR) that partitions active experts into top-score and low-score categories, aiming to replace low-score experts with those already in GPU memory. As shown in Fig. 18, the expert-cache router significantly reduces the probability that active low-score experts are not in GPU memory. Notably, its optimization effect grows more pronounced as the proportion of low-score experts in active experts increases (*i.e.*, as the low ratio rises).

Expert prefetching analysis. Compared to +CR, this reduces TPOT by 14% and increases GPU cache ratio by 12%. We incorporate online prefetching (labeled +Pre), which prefetches top-score experts before computation. These first three steps significantly reduce TPOT by improving the GPU expert cache ratio: as shown in Fig. 18, the number of top-score active experts requiring loading is drastically reduced via prefetching.

As depicted in Fig. 20, PCIe load time increases with prefetching, yet TPOT decreases. The increased PCIe load time results from occasional misidentification of low-score experts as top-score ones, leading to unnecessary fetching. TPOT reduction is achieved by ensuring that prefetching-related PCIe loads overlap with computation, not interfering with current layer expert loading, thus maximizing prefetching benefits. Additionally, Fig. 21 indicates our prefetching method attains

an average accuracy of about 82% in predicting top-score experts. Even if a prefetched expert is not top-score, it has a 95% chance of being active, significantly outperforming methods based solely on residual results. Prefetching only these predicted top-score experts allows the router to utilize them directly, boosting efficiency.

CPU-assisted Task Load Scheduling. Compared to +Pre, this cuts TPOT by 34% on average but decreases GPU cache ratio by 3%. We integrate the CPU-assisted scheduling (labeled +BA) to balance CPU usage and load. The reduction in TPOT is achieved because CPU computation, though slower than GPU computation, overlaps with load time, allowing CPU assistance without additional overhead and maintaining high expert utilization in GPU memory.

A slight decrease in the GPU cache ratio occurs as experts that would typically be loaded are computed directly on the CPU, which slows GPU cache updates, yet high utilization is maintained. As shown in Fig. 22, the scheduling minimizes the difference between CPU and PCIe times, ensuring CPU computation is fully covered by PCIe load time and maximizing CPU computation’s supportive role. However, challenges remain: the significant disparity between CPU computation and load times, with CPU time in S3 being only one-third of load time, necessitates many experts for effective balancing. Additionally, the limited number of active experts in MoE per decoding step hinders consistent balancing of PCIe load and CPU computation times.

VII. RELATED WORK

This section introduces relevant MoE serving approaches. For accelerating MoE inference under GPU memory constraints, approaches are categorized into general LLM inference optimization and MoE-specific expert scheduling, with the latter further divided into online and offline MoE serving. **Online MoE Serving** manages dynamically changing workloads on edge devices, where loading or computing non-resident expert models significantly impacts inference latency. As workloads vary, the active expert set changes, necessitating experts not in GPU memory. These methods adapt via flexible expert scheduling to sustain acceleration effectiveness. MoE-Infinity [43] uses a request-level expert activation matrix for coarse offloading decisions, while HybriMoE [52] optimizes CPU computation, enhancing CPU-load pipelines to minimize the maximum of CPU and loading times.

Offline MoE Serving. Designed for predetermined workloads, these strategies include MoE-lightning’s CGOPipe—a CPU-GPU-I/O pipelining schedule with a performance model to maximize resource utilization [12]. MoE-Lightning relies heavily on prior knowledge of the workload’s average prompt length and generation length to search for its optimal scheduling policy. If this information is unavailable or deviates significantly from the actual workload, it becomes challenging to maintain a consistent micro-batch size across requests. Furthermore, unexpected variations in sequence lengths can lead to excessive CPU memory usage and KV cache transfer

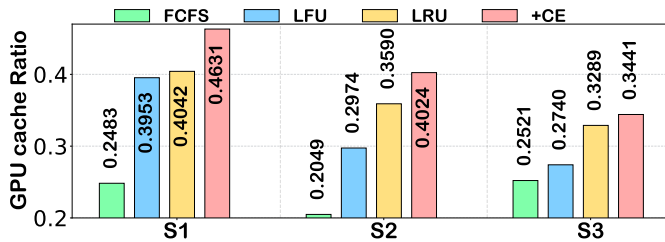


Fig. 19: “+CE” vs traditional cache methods.

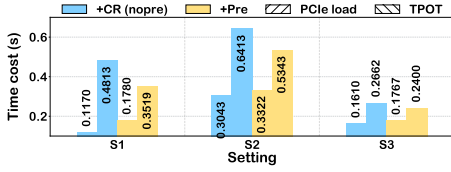


Fig. 20: PCIe time vs TPOT.

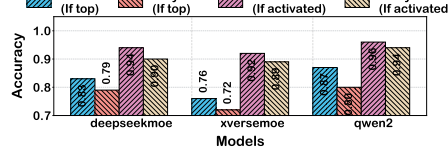


Fig. 21: Prefetching accuracy.

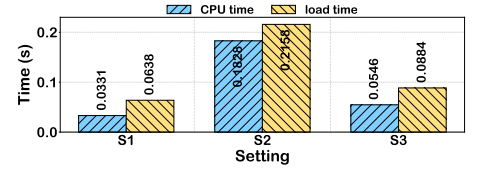


Fig. 22: CPU vs PCIe time.

TABLE VI: Accuracy Gains from Manually Lowered Scores.

Dataset	Original	Score-lower	SMoE($\alpha = 0.25$)
Math_II_MCQs (%)	71.56	74.44	72.94
History_MCQs (%)	80.84	83.97	83.90
Biology_MCQs (%)	81.33	82.64	87.33
race (%)	80.0	81.3	80.4
WiC (%)	60.5	60.7	60.7

overheads. Ultimately, this disrupts the optimized CPU-GPU pipeline, leading to a drastic reduction in overall throughput. Expert pruning [14] reduces memory usage by removing underutilized experts, tailoring resource management to specific workloads. Knowledge distillation [53] produces compact sparse MoE models; ComPEFT [45] demonstrates expert compression without accuracy loss, while MC-SMoE [31] further decomposes merged experts into low-rank and sparse alternatives. All these works address pre-known workloads. However, since we aim to handle unpredictable workloads on edge devices, we only compare with online MoE methods.

VIII. DISCUSSION

Expert substitution overcomes UMA constraints. Our expert substitution strategy remains directly applicable to Unified Memory Architectures (UMA) found in Apple M-series chips. Although UMA eliminates the discrete PCIe link between the CPU and GPU, the total system memory on edge devices is frequently insufficient to house the full parameters of large-scale MoE models. A typical Mac Studio equipped with 32GB or 64GB of memory cannot accommodate models requiring 100GB or more without heavily relying on SSD swapping, which inherently reverts to a PCIe bottleneck. In these memory-constrained scenarios, substituting low-score experts with those already cached in the unified memory pool largely reduces the frequency of high-latency storage accesses.

Accuracy Improvement Analysis. We attribute the accuracy improvement to the suppression of “noisy” activations, which occurs when the gating scores of low-score experts are significantly lower than those of top-score experts. To provide concrete evidence, we have conducted a validation experiment where we manually decreased the gating scores of these low-score experts without performing any expert substitution. As Table VI show, we observed similar accuracy gains across several datasets, which confirms the regularization effect in some cases. However, in the other cases low-score experts are close to the top-score experts, such lowering scores will lead to accuracy loss, which explains why methods like expert pruning suffer from accuracy loss. Since lowering scores alone

cannot speed up inference, expert substitution leverages this noise-filtering to increase GPU cache hit rates.

Design Shifts from kTransformer [1]. We did not build SMoE on kTransformer for three reasons: (1) Hardware portability: kTransformer relies on high-end CPU features (*e.g.*, AMX and AVX-512) often unavailable on edge devices. (2) Design strategy: kTransformer relies on CPU computation, while SMoE keeps experts GPU-resident with minimal CPU use, removing reliance on high-end CPU. In practice, edge devices rarely have extremely strong CPUs but weak GPUs. (3) Accuracy isolation: We use lossless inference to isolate the accuracy impact of our replacement strategy from quantization noise. SMoE is orthogonal to quantization and can integrate with frameworks like kTransformer.

Selection of Baselines & SOTA MoE Systems. These systems address different problems and are orthogonal to substitution. (1) D²MoE [41] involves mixed quantization and workload-based retraining; in contrast, SMoE is plug-and-play and reduces transfers. (2) ExpertFlow’s [21] routing predictors scale poorly with increasing domains and expert counts. (3) EC2MoE [46] optimizes for end-to-cloud bandwidth, unlike our PCIe-based environment.

IX. CONCLUSION

In conclusion, we present an approach for deploying MoE LLMs on edge devices by substituting low-importance active experts with functionally similar ones already cached in GPU memory, thereby preserving accuracy. We establish a robust CPU-GPU-load pipeline system named SMoE, providing an effective solution for LLM deployment on edge.

X. ACKNOWLEDGEMENTS

This work was supported in part by the National Key R&D Program of China under Grant No. 2023YFB4502400, in part by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under Grant No. JYB2025XDXM901, in part by the National Natural Science Foundation of China (NSFC) under Grants No. 62272223, U22A2031, and 62402212, U24B20153, in part by the Natural Science Foundation for Young Scientists of Jiangsu Province under Grant No. BK20241245, in part by the Collaborative Innovation Center of Novel Software Technology and Industrialization, Nanjing University, and in part by the Jiangsu High-level Innovation and Entrepreneurship (Shuangchuang) Program.

A. Abstract

This artifact provides the source code, scripts, and instructions to reproduce the core latency evaluations of SMoE, an algorithm-system co-design that pushes Mixture-of-Experts (MoE) to the edge via expert substitution. Specifically, this artifact focuses on reproducing the Time Per Output Token (TPOT) and GPU cache hit ratio metrics for the Qwen2-57B-A14B model.

B. Artifact check-list (meta-information)

- **Algorithm:** Expert Substitution, CPU-assisted task load scheduling.
- **Program:** SMoE Evaluation Scripts (Python).
- **Model:** Qwen2-57B-A14B-Instruct.
- **Dataset:** Gaokao, triviaqa, WiC, Race-mid, gsm8k.
- **Hardware:** Single NVIDIA A6000 GPU (48GB) on PCIe Gen 4 with Intel Xeon Gold 6444Y (Setting S3).
- **Metrics:** TPOT (Time Per Output Token) and GPU cache hit
- **Output:** Logs including TPOT results corresponding to Figure 12 and GPU cache hit corresponding to Figure 13.
- **Experiments:** Decoding performance evaluation at batch size = 1.
- **How much CPU memory required (approximately)?:** 150GB (The Qwen2-57B-A14B model alone requires 107GB)
- **Publicly available?:** Yes (<https://github.com/goingsshr/SMoE>).

C. Description

1) *How to access:* The source code and evaluation scripts are available at: <https://github.com/goingsshr/SMoE> or <https://doi.org/10.6084/m9.figshare.31982136>.

2) *Hardware dependencies:* To reproduce the Setting S3 experiments, a single NVIDIA A6000 GPU (48GB) is required. The system should utilize a PCIe 4.0 interface. As specified, the host system requires approximately 150GB of CPU memory to handle the model weights during the loading and offloading processes.

3) *Software dependencies:* Our implementation relies on Python and several specific packages. A complete list of dependencies and setup instructions is available in the repository’s `README.md`.

4) *Datasets:* The experiments utilize standard benchmarks: Gaokao, triviaqa, WiC, Race-mid, and gsm8k. Evaluation scripts will automatically download the required subsets.

5) *Models:* The Qwen2-57B-A14B-Instruct model can be downloaded from HuggingFace.

D. Installation

The complete source code and environment setup instructions are hosted in our GitHub repository. We delegate the step-by-step installation guide to the repository’s `README.md`.

E. Experiment workflow

The experiment workflow is fully automated via provided shell scripts, which trigger inference across the five datasets under specified batch sizes.

F. Evaluation and expected results

Executing the workflow generates comprehensive execution logs. The primary evaluation metrics are the average Time Per Output Token (TPOT) and the GPU cache hit ratio.

Our scripts automatically parse these logs to extract both metrics, which are expected to demonstrate significant latency reductions consistent with Figures 12 and 13 (Setting S3) in the paper. For a complete reproducibility guide—including script execution, output parsing, and mapping terminal results to the paper’s figures—please refer to the repository’s `README.md`.

G. Experiment customization

Users can modify the ‘-alpha’ parameter in the execution script to adjust the expert substitution threshold. For the Setting S3 reproduction, α is default to 0.25.

REFERENCES

- [1] “Ktransformers: A flexible framework for experiencing cutting-edge llm inference optimizations.” <https://github.com/kvccache-ai/KTransformers>, 2024.
- [2] “Llama.cpp: a c++ implementation enabling efficient llm inference on cpus.” <https://github.com/ggml-org/llama.cpp>, 2024.
- [3] “Opencompass is an llm evaluation platform, supporting a wide range of models (llama3, mistral, internlm2, gpt-4, llama2, qwen, glm, claude, etc) over 100+ datasets.” <https://github.com/open-compass/opencompass>, 2024.
- [4] “This url describes some of the common llm inference metrics.” <https://docs.nvidia.com/nim/benchmarking/llm/latest/metrics.html>, 2024.
- [5] “This url introduces the llm deepseek-moe-16b-chat.” <https://huggingface.co/deepseek-ai/deepseek-moe-16b-chat>, 2024.
- [6] “This url introduces the llm qwen2-72b-a14b-instruct.” <https://huggingface.co/Qwen/Qwen2-72B-A14B-Instruct>, 2024.
- [7] “This url introduces the llm /xverse-moe-a4.2b-chat.” <https://huggingface.co/xverse/XVERSE-MoE-A4.2B-Chat>, 2024.
- [8] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley *et al.*, “DeepSpeed-inference: enabling efficient inference of transformer models at unprecedented scale,” in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–15.
- [9] R. Awasthi, S. P. Ramachandran, S. Mishra, D. Mahapatra, H. Arshad, A. Atreja, A. Bhattacharyya, A. Bhattad, N. Singh, J. B. Cywinski *et al.*, “Artificial intelligence in healthcare: 2024 year in review,” *medRxiv*, pp. 2025–02, 2025.
- [10] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi, “Piqa: Reasoning about physical commonsense in natural language,” in *Thirty-Fourth AAAI Conference on Artificial Intelligence*, 2020.
- [11] W. Cai, J. Jiang, F. Wang, J. Tang, S. Kim, and J. Huang, “A survey on mixture of experts in large language models,” *IEEE Transactions on Knowledge and Data Engineering*, 2025.
- [12] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, “Moe-lightning: High-throughput moe inference on memory-constrained gpus,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 715–730.
- [13] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, “Evaluating large language models trained on code,” 2021.
- [14] T. Chen, S. Huang, Y. Xie, B. Jiao, D. Jiang, H. Zhou, J. Li, and F. Wei, “Task-specific expert pruning for sparse mixture-of-experts,” *arXiv preprint arXiv:2206.00277*, 2022.
- [15] Z. Chkribene, R. Hamila, A. Gouisse, and U. Devrim, “Large language models (llm) in industry: A survey of applications, challenges, and trends,” in *2024 IEEE 21st International Conference on Smart Communities: Improving Quality of Life using AI, Robotics and IoT (HONET)*. IEEE, 2024, pp. 229–234.
- [16] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, “Think you have solved question answering? try arc, the ai2 reasoning challenge,” *arXiv:1803.05457v1*, 2018.
- [17] D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu *et al.*, “Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models,” *arXiv preprint arXiv:2401.06066*, 2024.
- [18] U. Deshpande, T. Janssen, M. Srivatsa, and S. Sundararaman, “Moesaic: Shared mixture of experts,” in *Proceedings of the 2024 ACM Symposium on Cloud Computing*, November 2024, pp. 434–442.
- [19] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [20] O. Friha, M. A. Ferrag, B. Kantarci, B. Cakmak, A. Ozgun, and N. Ghoualmi-Zine, “Llm-based edge intelligence: A comprehensive survey on architectures, applications, security and trustworthiness,” *IEEE Open Journal of the Communications Society*, 2024.
- [21] X. He, S. Zhang, Y. Wang, H. Yin, Z. Zeng, S. Shi, Z. Tang, X. Chu, I. Tsang, and O. Y. Soon, “Expert-flow: Optimized expert activation and token allocation for efficient mixture-of-experts inference,” *arXiv preprint arXiv:2410.17954*, 2024.
- [22] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- [23] X. Jiang, Y. Zhou, S. Cao, I. Stoica, and M. Yu, “Neo: Saving gpu memory crisis with cpu offloading for online llm inference,” *arXiv preprint arXiv:2411.01142*, 2024.
- [24] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer, “triviaqa: A Large Scale Distantly Supervised Challenge

- Dataset for Reading Comprehension,” *arXiv e-prints*, p. arXiv:1705.03551, 2017.
- [25] H. Kang, Z. Yu, and C. Xiong, “Flame-moe: A transparent end-to-end research platform for mixture-of-experts language models,” *arXiv preprint arXiv:2505.20225*, 2025.
 - [26] H. Kim, N. Wang, Q. Xia, J. Huang, A. Yazdanbakhsh, and N. S. Kim, “Lia: A single-gpu llm inference acceleration with cooperative amx-enabled cpu-gpu computation and cxl offloading,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, June 2025, pp. 544–558.
 - [27] R. Kong, Y. Li, W. Wang, L. Kong, and Y. Liu, “Serving moe models on resource-constrained edge devices via dynamic expert swapping,” *IEEE Transactions on Computers*, 2025.
 - [28] G. Lai, Q. Xie, H. Liu, Y. Yang, and E. Hovy, “RACE: Large-scale ReAding comprehension dataset from examinations,” in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. Copenhagen, Denmark: Association for Computational Linguistics, Sep. 2017, pp. 785–794. [Online]. Available: <https://aclanthology.org/D17-1082>
 - [29] C. Li, Y. Yin, X. Wu, J. Zhu, Z. Gao, D. Niu, Q. Wu, X. Si, Y. Xie, C. Zhang *et al.*, “H2-llm: Hardware-dataflow co-exploration for heterogeneous hybrid-bonding-based low-batch llm inference,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 194–210.
 - [30] N. Li, S. Guo, T. Zhang, M. Li, Z. Hong, Q. Zhou, X. Yuan, and H. Zhang, “The moe-empowered edge llms deployment: Architecture, challenges, and opportunities,” *arXiv preprint arXiv:2502.08381*, 2025.
 - [31] P. Li, Z. Zhang, P. Yadav, Y.-L. Sung, Y. Cheng, M. Bansal, and T. Chen, “Merge, then compress: Demystify efficient smoe with hints from its routing policy,” *arXiv preprint arXiv:2310.01334*, 2023.
 - [32] X. Lu, Q. Liu, Y. Xu, A. Zhou, S. Huang, B. Zhang, J. Yan, and H. Li, “Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models,” *arXiv preprint arXiv:2402.14800*, 2024.
 - [33] M. T. Pilehvar and J. Camacho-Collados, “Wic: the word-in-context dataset for evaluating context-sensitive meaning representations,” *arXiv preprint arXiv:1808.09121*, 2018.
 - [34] D. Rivkin, F. Hogan, A. Feriani, A. Konar, A. Sigal, X. Liu, and G. Dudek, “Aiot smart home via autonomous llm agents,” *IEEE Internet of Things Journal*, 2024.
 - [35] R. Sarkar, H. Liang, Z. Fan, Z. Wang, and C. Hao, “Edge-moe: Memory-efficient multi-task vision transformer architecture with task-level sparsity via mixture-of-experts,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, October 2023, pp. 1–9.
 - [36] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. V. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” *arXiv preprint arXiv:1701.06538*, 2017.
 - [37] R. Singh and S. S. Gill, “Edge ai: a survey,” *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023.
 - [38] X. Song, Z. Zhong, R. Chen, and H. Chen, “Promoe: Fast moe-based llm serving using proactive caching,” *arXiv preprint arXiv:2410.22134*, 2024.
 - [39] Y. Song, Z. Mi, H. Xie, and H. Chen, “Powerinfer: Fast large language model serving with a consumer-grade gpu,” in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024, pp. 590–606.
 - [40] B. Spector and C. Re, “Accelerating llm inference with staged speculative decoding,” *arXiv preprint arXiv:2308.04623*, 2023.
 - [41] H. Wang, Q. Zhou, Z. Hong, and S. Guo, “D2moe: Dual routing and dynamic scheduling for efficient on-device moe-based llm serving,” in *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*, 2025, pp. 574–588.
 - [42] W. Wang, L. Mi, S. Cen, H. Dai, Y. Li, X. Fu, and Y. Liu, “Region-based content enhancement for {Efficient} video analytics at the edge,” in *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, 2025, pp. 613–633.
 - [43] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, “Moe-infinity: Activation-aware expert offloading for efficient moe serving,” *arXiv e-prints*, pp. arXiv-2401, 2024.
 - [44] Z. Xue, Y. Song, Z. Mi, X. Zheng, Y. Xia, and H. Chen, “Powerinfer-2: Fast large language model inference on a smartphone,” *arXiv preprint arXiv:2406.06282*, 2024.
 - [45] P. Yadav, L. Choshen, C. Raffel, and M. Bansal, “Compeft: Compression for communicating parameter efficient updates via sparsification and quantization,” *arXiv preprint arXiv:2311.13171*, 2023.
 - [46] Z. Yang, Y. Hu, S. Sun, and W. Ji, “Ec2moe: Adaptive end-cloud pipeline collaboration enabling scalable mixture-of-experts inference,” *arXiv preprint arXiv:2508.06024*, 2025.
 - [47] Z. Yu, S. Liang, T. Ma, Y. Cai, Z. Nan, D. Huang, T. Chen *et al.*, “Cambricon-llm: A chiplet-based hybrid architecture for on-device inference of 70b llm,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, November 2024, pp. 1474–1488.
 - [48] D. Zhang, H. Zheng, W. Yue, and X. Wang, “Advancing its applications with llms: A survey on traffic management, transportation safety, and autonomous driving,” in *International Joint Conference on Rough Sets*. Springer, 2024, pp. 295–309.
 - [49] M. Zhang, X. Shen, J. Cao, Z. Cui, and S. Jiang, “Edgeshard: Efficient llm inference via collaborative edge computing,” *IEEE Internet of Things Journal*, 2024.
 - [50] Y. Zhang, S. Aggarwal, and T. Mitra, “Daop: Data-aware

offloading and predictive pre-calculation for efficient moe inference,” in *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, March 2025, pp. 1–7.

- [51] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging llm-as-a-judge with mt-bench and chatbot arena,” 2023.
- [52] S. Zhong, Y. Sun, L. Liang, R. Wang, R. Huang, and M. Li, “Hybrimoe: Hybrid cpu-gpu scheduling and cache management for efficient moe inference,” *arXiv preprint arXiv:2504.05897*, 2025.
- [53] T. Zhong, Z. Chi, L. Gu, Y. Wang, Y. Yu, and J. Tang, “Meta-dmoe: Adapting to domain shift by meta-distillation from mixture-of-experts,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 22 243–22 257, 2022.